REST ASSURED
CLASS 6

POJO Classes • Getters & Setters • Lombok
Serialization • Nested JSON • Request Payload Design

Beginner-friendly explanations, runnable examples, real-world scenarios, troubleshooting, hands-on exercises, and industry best practices

What this workbook is based on
Your uploaded Class 6 notes cover POJO request payloads, getter/setter generation, Lombok annotations, Posts and Comments APIs, nested Product/Rating objects, and nested Users/Address/Geolocation objects. This workbook follows those topics, explains the code from scratch, and flags important corrections and best-practice improvements.

Study principle: understand the object first, then watch how it becomes JSON.

How to use these notes

Read each concept, type the examples into your own Maven project, run them, inspect the serialized JSON, and then complete the exercises without copying the solution first. The examples use tutorial APIs from your notes; always confirm the contract and response codes of the API you are testing.

Learning path

- Understand JSON request payloads and why we use POJO classes.
- Create a Java POJO with private fields, setters, and getters.
- Pass a POJO into Rest Assured body() and understand serialization.
- Use Lombok to reduce repetitive getter/setter code.
- Model nested JSON for Products/Rating and Users/Address/Geolocation.
- Add meaningful response assertions and diagnose common failures.
- Apply naming, type, security, and maintainability best practices.

Contents

- 1. Request payloads and POJO fundamentals
- 2. Build a Posts POJO manually
- 3. Send a POJO using Rest Assured
- 4. Serialization and object mappers
- 5. Generate getters/setters: IDE and Lombok
- 6. Comments POJO and practical use case
- 7. Nested JSON: Products and Rating
- 8. Nested JSON: Users, Name, Address, Geolocation
- 9. Assertions and robust test design
- 10. Troubleshooting and common mistakes
- 11. Industry best practices
- 12. Hands-on exercises and answer key
- 13. Interview revision questions and checklist

1. Request payloads and POJO fundamentals

1.1 What is a request payload?

A request payload is the data sent by a client to an API, commonly in the body of a POST, PUT, or PATCH request. For example, when a shopping website creates a product, the client may send its title, price, description, category, and image URL as JSON.

Example JSON payload

```
{
  "userId": 1,
  "title": "E-commerce launch",
  "body": "Products are now available"
}
```

The JSON keys (userId, title, body) and their value types form part of the API contract. Swagger/OpenAPI documentation, an API schema, or a known request/response example should guide the Java class you create.

1.2 What does POJO mean?

POJO means Plain Old Java Object. It is a regular Java class used to hold data. A POJO commonly contains private fields, getters, setters, and sometimes constructors. It does not need to extend a special framework class.

JSON field	Java field type	Example value	Why the type matters
userId	int or Integer	1	Numeric identifier; Integer can also represent null.
title	String	E-commerce launch	Text values are represented as strings.
body	String	Products are available	Long text content.
price	BigDecimal	299.99	Prefer precise decimal handling for money.
rating	Rating	{rate: 4.5, count: 20}	Nested JSON object becomes a nested Java object.

1.3 Why use a POJO instead of writing raw JSON?

Raw JSON string	POJO object
Easy for a tiny one-off request, but quote/escape errors are common.	Java compiler helps catch spelling and type mistakes.
Changing test data may mean editing a long string.	Set different values with methods such as setTitle().
Nested JSON strings can become hard to maintain.	Nested objects can model nested JSON naturally.
Data may be duplicated across tests.	Reusable request objects support cleaner test data factories.

Real-world scenario: product onboarding
A QA engineer tests the Create Product API every build. Instead of maintaining ten raw JSON strings, the engineer creates a ProductRequest POJO and varies title, price, category, and rating data in each test.

2. Build a Posts POJO manually

Your source creates a Posts class with userId, title, and body. The corrected version below uses private fields and follows JavaBean naming conventions.

Posts.java

```
package tutorial2;

public class Posts {
    private int userId;
    private String title;
    private String body;

    public int getUserId() {
        return userId;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getBody() {
        return body;
    }

    public void setBody(String body) {
        this.body = body;
    }
}
```

2.1 Explanation of the code

- **private int userId;** Declares a field that stores the user identifier. private restricts direct access from unrelated classes.
- **private String title;** Declares text data for the post title.
- **getUserId()** A getter returns the current value stored in userId.
- **setUserId(int userId)** A setter receives a value and saves it in the field.
- **this.userId** = **userId;** The left side is the object field; the right side is the method parameter. this makes the distinction explicit.
- **getBody()** Uses the conventional get + field name pattern. Your source had body() in one version; use getBody() for consistent JavaBean introspection.

2.2 Why private fields?

Encapsulation means the class controls how its internal data is accessed. Private fields prevent other classes from changing the fields directly and allow values to be validated in setter methods if needed.

Optional validation inside a setter

```
public void setTitle(String title) {
    if (title == null || title.isBlank()) {
        throw new IllegalArgumentException("title is required");
    }
    this.title = title;
}
```

This kind of validation is useful for enforcing basic object rules, but it does not replace server-side API validation. Many request DTOs keep setters simple and validate input with Bean Validation or test assertions instead.

2.3 Object creation and setting values

Create and populate the POJO

```
Posts posts = new Posts();
posts.setUserId(1001);
posts.setTitle("Ecommerce Posts");
posts.setBody("All ecommerce products are available");
```

new Posts() creates an object in memory. Each setter fills one field. At this point, the data is still a Java object; it has not yet been sent over HTTP.

3. Send a POJO using Rest Assured

3.1 Complete test example

PostsEndpointTest.java

```
package tutorial2;

import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import org.testng.annotations.Test;

import static org.hamcrest.Matchers.equalTo;
import static org.testng.Assert.assertTrue;

public class PostsEndpointTest {

    @Test
    public void createPostUsingPojo() {
        Posts payload = new Posts();
        payload.setUserId(1);
        payload.setTitle("E-commerce launch");
        payload.setBody("Products are now available");

        Response response = RestAssured.given()
            .baseUri("https://jsonplaceholder.typicode.com")
            .contentType(ContentType.JSON)
            .accept(ContentType.JSON)
            .body(payload)
            .when()
            .post("/posts");

        response.then()
            .log().ifValidationFails()
            .statusCode(201)
            .body("userId", equalTo(1))
            .body("title", equalTo("E-commerce launch"))
            .body("body", equalTo("Products are now available"));

        int generatedId = response.jsonPath().getInt("id");
        assertTrue(generatedId > 0, "API should return a positive id");
    }
}
```

3.2 Line-by-line explanation

Code	Meaning
@Test	TestNG discovers and runs this method as a test.

Code	Meaning
RestAssured.given()	Starts building an HTTP request.
.baseUri(...)	Sets the common server address.
.contentType(ContentType.JSON)	Tells the server that the request body is JSON.
.accept(ContentType.JSON)	Requests a JSON response from the server.
.body(payload)	Supplies the Java object. Rest Assured serializes it to JSON when a supported object mapper is on the classpath.
.post("/posts")	Sends an HTTP POST to the endpoint.
.statusCode(201)	Asserts that the API returned Created. Use the code required by your API contract.
.body("title", equalTo(...))	Checks a value extracted from the JSON response.
response.jsonPath().getInt("id")	Reads the id field as an integer.
assertTrue(generatedId > 0, ...)	Adds a readable validation of the generated id.

Important note about JSONPlaceholder
JSONPlaceholder is a practice API. Its POST endpoint normally simulates creation and returns a representative response; do not assume a new record is permanently stored. In your own backend, use the API contract to determine the exact response code and persistence behaviour.

3.3 What is request content type?

Content-Type describes the format of the request body. Accept describes the format the client wants back. For JSON APIs, application/json is common for both, but they serve different purposes.

```
given()
    .contentType(ContentType.JSON)   // format we send
    .accept(ContentType.JSON)        // format we want back
    .body(payload)
.when()
    .post("/posts");
```

4. Serialization and object mappers

4.1 What is serialization?

Serialization converts a Java object into a representation suitable for transport, such as JSON. Deserialization is the reverse: reading JSON and converting it into a Java object. Rest Assured delegates object conversion to a supported mapper such as Jackson or Gson when configured on the classpath.

Conceptual flow

```
Java object                    JSON sent over HTTP
Posts                        {
  userId = 1       ----->        "userId": 1,
  title = "Hello"                "title": "Hello",
  body = "World"                    "body": "World"
                             }
```

4.2 What dependency may be needed?

If Rest Assured cannot determine how to serialize your POJO, add a supported object-mapper library such as Jackson Databind to your Maven project. Keep dependency versions compatible with the Java version and the rest of your project.

pom.xml example

```
<!-- Example dependency; align versions with your project -->
<dependency>
   <groupId>com.fasterxml.jackson.core</groupId>
   <artifactId>jackson-databind</artifactId>
   <version>2.17.2</version>
</dependency>
```

Typical symptoms of a mapper/configuration problem include an exception saying no serializer is found or a request body that is not the JSON shape you expected. Inspect a safe test log or the request body in a debugger.

4.3 Field names and JSON names

Jackson typically maps JavaBean getters/fields to JSON property names. If the API uses a different name from your Java variable, use an explicit mapping annotation such as @JsonProperty.

Mapping the JSON key "long"

```
import com.fasterxml.jackson.annotation.JsonProperty;

public class Geolocation {
   private String lat;

   @JsonProperty("long")
   private String geoLong;

   public String getLat() { return lat; }
   public void setLat(String lat) { this.lat = lat; }
   public String getGeoLong() { return geoLong; }
   public void setGeoLong(String geoLong) { this.geoLong = geoLong; }
}
```

Java has a primitive type named long, so long cannot be used as a Java variable name. geoLong is a legal Java name; @JsonProperty("long") tells Jackson to emit/read the JSON key long.

5. Generate getters and setters: IDE and Lombok

5.1 Eclipse generation

1. Open the POJO class in Eclipse.
2. Right-click in the editor and choose Source.
3. Choose Generate Getters and Setters.
4. Select the fields you need, then generate the methods.
5. Review the generated methods before committing the class.

This is a good option when you want code that does not require an annotation processor or additional library.

5.2 Lombok annotations

Lombok generates common Java boilerplate at compile time. Your source shows @Getter, @Setter, and @Data.

Annotation	Typical generated code
@Getter	Getter methods for fields.
@Setter	Setter methods for fields.
@NoArgsConstructor	No-argument constructor.
@AllArgsConstructor	Constructor with all fields.
@ToString	toString() method.
@EqualsAndHashCode	equals() and hashCode().
@Data	A bundle including getters, setters, toString, equals/hashCode, and a required-arguments constructor.

Lombok alternative to manual getters/setters

```
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class Posts {
   private int userId;
   private String title;
   private String body;
}
```

With the annotations above, you do not write each getter/setter manually. The Java source is shorter, but your build still needs Lombok configured. IDE support or annotation-processing settings may also be needed so the editor recognises generated methods.

Security caution with @Data
@Data generates toString(), which can expose every field. Do not casually use it on objects containing passwords, access tokens, API secrets, or personal data. Prefer @Getter/@Setter or a carefully controlled toString() for sensitive request objects.

5.3 Lombok Maven dependency

Dependency example from the class notes

```
<dependency>
   <groupId>org.projectlombok</groupId>
   <artifactId>lombok</artifactId>
   <version>1.18.48</version>
   <scope>provided</scope>
</dependency>
```

Use a version that is compatible with your JDK and build tooling. If code compiles in Maven but the IDE shows red errors, check the IDE Lombok support and annotation-processing configuration.

6. Comments POJO: a second practical use case

The source models a comment with postId, name, email, and body. This resembles a comment/review form: a user submits a comment associated with a parent post. The object keeps those related values together.

Comments.java

```
package tutorial3;

import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class Comments {
    private int postId;
    private String name;
    private String email;
    private String body;
}
```

CommentsTest.java

```
package tutorial3;

import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import org.testng.annotations.Test;

import static org.hamcrest.Matchers.equalTo;

public class CommentsTest {
    @Test
    public void createComment() {
        Comments payload = new Comments();
        payload.setPostId(1);
        payload.setName("Rohini");
        payload.setEmail("rohini@example.com");
        payload.setBody("Automation code review");

        Response response = RestAssured.given()
            .baseUri("https://jsonplaceholder.typicode.com")
            .contentType(ContentType.JSON)
            .accept(ContentType.JSON)
            .body(payload)
            .when()
            .post("/comments");

        response.then()
            .log().ifValidationFails()
            .statusCode(201)
            .body("postId", equalTo(1))
            .body("name", equalTo("Rohini"))
            .body("email", equalTo("rohini@example.com"));
    }
}
```

What to learn from this example

- One POJO represents one coherent request shape.
- The test data is easy to vary: use a different postId, name, email, or body for positive and negative cases.

- Use safe example email addresses in training material. Never use real customer information in logs or public repositories.
- A successful 201 response is not enough by itself; verify that the returned values match the intended request and contract.

7. Nested JSON: Products and Rating

7.1 Why is a nested class needed?

A product API may have an object inside another object. In the source, the Products object has a rating object, and rating contains rate and count. If you model rating as a string, the JSON shape will be wrong: it needs to be an object with its own properties.

Expected nested JSON shape

```
{
  "title": "Laptop Sales",
  "price": 2300.45,
  "description": "Refurbished laptops available",
  "category": "Electronics",
  "image": "https://example.test/laptop.png",
  "rating": {
    "rate": 4.5,
    "count": 200
  }
}
```

7.2 POJO classes

ProductRequest.java

```
import java.math.BigDecimal;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class ProductRequest {
    private String title;
    private BigDecimal price;
    private String description;
    private String category;
    private String image;
    private Rating rating;
}
```

Rating.java

```
import java.math.BigDecimal;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class Rating {
    private BigDecimal rate;
    private Integer count;
}
```

The product contains a Rating field. Because Rating is a Java object, Jackson can serialize it as the nested JSON object shown above.

7.3 Build the nested object and send it

Product request example

```
import java.math.BigDecimal;
import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;

Rating rating = new Rating();
rating.setRate(new BigDecimal("4.5"));
rating.setCount(200);

ProductRequest product = new ProductRequest();
product.setTitle("Laptop Sales");
product.setPrice(new BigDecimal("2300.45"));
product.setDescription("Refurbished laptops available");
product.setCategory("Electronics");
product.setImage("https://example.test/laptop.png");
product.setRating(rating);

Response response = RestAssured.given()
   .baseUri("https://fakestoreapi.com")
   .contentType(ContentType.JSON)
   .accept(ContentType.JSON)
   .body(product)
   .when()
   .post("/products");

response.then()
   .log().ifValidationFails()
   .statusCode(200); // Verify expected code for the API contract
```

Why BigDecimal for price?
float and double use binary floating-point and can introduce rounding surprises. BigDecimal is a better default for money. Construct it from a String, such as new BigDecimal("2300.45"), rather than from a double literal.

7.4 Align the class with the API contract

The provided source includes an email field in one product example. A typical product contract may not accept email for a product. Do not add fields simply because another DTO has them. Match the Swagger/OpenAPI request schema. If the server rejects unknown fields, an extra property can fail the request; even if it silently ignores it, the test may be misleading.

8. Nested JSON: Users, Name, Address, Geolocation

8.1 Understand the shape first

Illustrative nested user payload

```
{
  "email": "kajal@example.com",
  "username": "kajal",
  "password": "<test-secret>",
  "name": {
    "firstname": "Kajal",
    "lastname": "Jadhav"
  },
  "phone": "9090901122",
  "address": {
    "geolocation": {
      "lat": "23.34",
      "long": "23"
    },
    "city": "Pune",
    "street": "Deccan",
    "number": 100,
    "zipcode": "122344"
  }
}
```

This JSON has two nested levels: name contains first and last name; address contains geolocation. Create one Java class for each reusable nested object.

8.2 Name and Geolocation

Name.java

```
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class Name {
    private String firstname;
    private String lastname;
}
```

Geolocation.java

```
import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class Geolocation {
    private String lat;

    @JsonProperty("long")
    private String geoLong;
}
```

8.3 Address class

Address.java

```
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class Address {
    private Geolocation geolocation;
    private String city;
    private String street;
    private Integer number;
    private String zipcode;
}
```

8.4 User request class

UserRequest.java

```
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class UserRequest {
    private String email;
    private String username;
    private String password;
    private Name name;
    private String phone;
    private Address address;
}
```

8.5 Build the graph of objects

Build parent and child POJOs

```
Name name = new Name();
name.setFirstname("Kajal");
name.setLastname("Jadhav");

Geolocation location = new Geolocation();
location.setLat("23.34");
location.setGeoLong("23");

Address address = new Address();
address.setGeolocation(location);
address.setCity("Pune");
address.setStreet("Deccan");
address.setNumber(100);
address.setZipcode("122344");

UserRequest user = new UserRequest();
user.setAddress(address);
user.setName(name);
user.setEmail("kajal@example.com");
user.setUsername("kajal");
String testPasswordFromEnvironment = System.getenv("TEST_USER_PASSWORD");
if (testPasswordFromEnvironment == null || testPasswordFromEnvironment.isBlank()) {
    throw new IllegalStateException("Set TEST_USER_PASSWORD environment variable");
}
user.setPassword(testPasswordFromEnvironment);
user.setPhone("9090901122");
```

The line user.setAddress(address) does not copy text from the address; it stores a reference to the Address object. During serialization, the object mapper reads the nested properties and creates the nested JSON structure.

Important: source-specific field __v
The uploaded source includes a __v field in Users. Some APIs use __v for an internal version counter. Do not send server-managed fields when creating a record unless the request contract explicitly requires them. If you must map the JSON key __v, use a normal Java name such as version and @JsonProperty("__v").

Map a special JSON property name

```
import com.fasterxml.jackson.annotation.JsonProperty;

@JsonProperty("__v")
private Integer version;
```

8.6 Real-world scenario: account registration

Imagine testing a delivery application sign-up API. The request contains account details, display name, phone number, address, and location. A nested POJO model lets one test create different cities and coordinates without manually editing JSON strings. Test both a complete valid address and cases where required nested values are missing.

9. Assertions and robust test design

A test that only prints the response is a demonstration, not a complete automated test. Assertions make the test fail when the actual behaviour differs from the expected behaviour.

9.1 Useful assertions

Response assertions

```
response.then()
   .log().ifValidationFails()
   .statusCode(201)
   .contentType(ContentType.JSON)
   .body("title", equalTo("E-commerce launch"))
   .body("userId", equalTo(1));
```

Validation	Example reason
Status code	Creation should return the contract-defined success code.
Content-Type	Response should be JSON when that is the documented format.
Returned fields	Returned title or email should match expected values.
Generated identifier	The response should contain a non-null, valid id if the contract promises one.
Nested field	rating.count or address.city should have the expected value.
Negative case	An invalid email or missing required field should return the documented client error.

9.2 Inspecting nested values

Nested JSONPath assertions

```
response.then()
   .body("rating.rate", equalTo(4.5F))
   .body("rating.count", equalTo(200))
   .body("category", equalTo("Electronics"));
```

JSONPath uses dot notation to access nested properties. For example, rating.count means find the rating object, then read its count property. Ensure the numeric types in your matcher align with how the JSON parser represents them; for precise decimals, consider reading BigDecimal or using suitable numeric matchers.

9.3 Separate request DTOs from response DTOs

For a very small tutorial, one POJO can be enough. In a larger framework, consider separate classes such as CreateProductRequest and ProductResponse. The request may omit server-generated id and timestamps, while the response includes them. Separating DTOs prevents accidentally sending read-only response fields back to the server.

10. Troubleshooting and common mistakes

Symptom	Likely cause	What to check
Request body is empty or malformed	Object mapper missing or field access/naming mismatch.	Check mapper dependency, getters, field names, Content-Type, and request logs with secrets masked.
Property appears as null or is missing	Getter name is unconventional, property name mismatch, or value was never set.	Use getBody(), inspect setter calls, and compare output JSON to the contract.
Nested object serializes incorrectly	Child field is a String or not populated.	Use a child POJO such as Rating rating; build and assign the child object.
Java compile error on long field	long is a Java keyword.	Use geoLong and map it with @JsonProperty("long").
Wrong number type or rounding	float/double for money or matcher type mismatch.	Use BigDecimal for currency and confirm JSON numeric types.
API rejects unknown fields	POJO contains fields not allowed by the request schema.	Remove unsupported fields; follow Swagger/OpenAPI.
Test passes but data is absent later	The practice API simulates writes or uses temporary data.	Verify persistence against your own test environment/database/API contract.
Lombok getters not recognised	IDE or annotation processing configuration.	Check Maven build, Lombok IDE support, JDK compatibility, and clean/rebuild.
Sensitive data appears in output	request.log().all() or @Data toString() exposes fields.	Remove full logging, mask secrets, and avoid toString() on sensitive DTOs.

10.1 Corrections to remember from the source

- Use getBody(), not body(), for a conventional getter.
- Fix the source typo repsonse to response and similarly check all variable spellings.
- Use a complete endpoint URL or configure baseUri before calling post().
- Add assertions rather than only response.then().log().all().
- Use a request class matching the documented schema; do not include an email field in ProductRequest unless the API supports it.
- Avoid hard-coding personal-looking data or passwords in shared code. Use synthetic data and environment-based secrets.

11. Industry best practices

11.1 Model the API contract, not assumptions

- Read Swagger/OpenAPI or the API specification before creating a POJO.
- Check exact property names, required fields, nullable fields, collection types, formats, and nested object structure.
- Keep request DTOs limited to fields accepted by that endpoint.

11.2 Choose data types deliberately

- Use String for text, Integer/Long where nullability matters, and BigDecimal for currency values.
- Use List<T> for JSON arrays and child POJOs for nested objects.
- Treat phone numbers and postcodes as strings if leading zeros or formatting must be preserved.

11.3 Make tests readable and deterministic

- Use descriptive test names such as createProduct_withValidPayload_returnsCreatedProduct.
- Assert status code, key response fields, and required behaviour.
- Prefer reusable payload builders or factory methods for repeated test data.
- Use unique email addresses or identifiers where duplicate constraints apply.
- Clean up created data where the test environment supports reliable cleanup.

11.4 Handle sensitive fields safely

- Do not hard-code production passwords, API keys, or tokens in source code.
- Avoid logging request/response bodies that contain passwords, personal data, or access tokens.
- Be careful with Lombok @Data because its generated toString() can include all fields.

11.5 Keep DTOs maintainable

- Prefer private fields and consistent naming conventions.
- Use @JsonProperty for unavoidable JSON/Java naming differences.
- Do not create one enormous class for unrelated endpoints; use focused request and response models.
- Review Lombok-generated behaviour before using @Data on classes with sensitive fields or complex equality semantics.

11.6 Sample Maven dependencies

Illustrative pom.xml dependencies

```
<dependencies>
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>5.5.0</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.10.2</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.2</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.48</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

Version note
The versions above are illustrative dependency coordinates for a learning project, not a guarantee that they are the newest versions. Use versions approved for your Java/JDK and existing project. Remove Lombok if you prefer manually generated getters and setters.

12. Hands-on exercises

Try each task before checking the answer key. Create a TestNG test class and POJOs in separate files when the example uses multiple public classes.

Exercise 1 — Basic POJO

Create a Book class with private fields id (int), title (String), and author (String), plus getters and setters.

Exercise 2 — Serialization

Create a Posts object with userId=5, title="API testing", and body="POJO practice". Send it as JSON using Rest Assured.

Exercise 3 — Assertions

Add assertions for status code 201 and returned title/userId.

Exercise 4 — Comments

Create a Comments POJO and POST postId, name, email, and body. Assert that the response email is correct.

Exercise 5 — Nested product

Create ProductRequest and Rating classes. Serialize rating.rate=4.5 and rating.count=100 as a nested JSON object.

Exercise 6 — Correct currency

Change the product price from float to BigDecimal and initialize it using a String.

Exercise 7 — Nested user

Create UserRequest, Name, Address, and Geolocation classes. Build a user object with city and coordinates.

Exercise 8 — JSON key long

Map the Java field geoLong to JSON property "long" using @JsonProperty.

Exercise 9 — Lombok

Replace manual getters/setters with @Getter and @Setter. Confirm the Maven build compiles.

Exercise 10 — Negative test

Send a request with a missing required title and assert the documented validation status and error response.

Exercise 11 — Data factory

Write a method createValidPost() that returns a new populated Posts object. Use it in two tests with different titles.

Exercise 12 — Safe logging

Replace full request logging with failure-only logging and ensure no password is printed.

Answer key and reference solutions

Solution 1 — Basic Book POJO

```
public class Book {
    private int id;
    private String title;
    private String author;

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
}
```

Solution 2 and 3 — Build and test a post

```
Posts post = new Posts();
post.setUserId(5);
post.setTitle("API testing");
post.setBody("POJO practice");

Response response = RestAssured.given()
   .baseUri("https://jsonplaceholder.typicode.com")
   .contentType(ContentType.JSON)
   .accept(ContentType.JSON)
   .body(post)
   .when()
   .post("/posts");

response.then()
   .log().ifValidationFails()
   .statusCode(201)
   .body("userId", equalTo(5))
   .body("title", equalTo("API testing"));
```

Solution 5 — Nested object

```
Rating rating = new Rating();
rating.setRate(new BigDecimal("4.5"));
rating.setCount(100);

ProductRequest product = new ProductRequest();
product.setTitle("Headphones");
product.setPrice(new BigDecimal("79.99"));
product.setRating(rating);
// Fill the other required fields according to your API schema.
// Pass product to .body(product).
```

Solution 8 — JSON property mapping

```
@JsonProperty("long")
private String geoLong;
```

Solution 11 — Data factory

```
public Posts createValidPost() {
   Posts post = new Posts();
   post.setUserId(1);
   post.setTitle("Default test title");
   post.setBody("Default test body");
   return post;
}
```

For exercises where the exact expected status or error body depends on the target API, inspect its OpenAPI contract or known API behaviour first. Do not copy an arbitrary expected status from a different service.

13. Interview revision questions

What is a POJO? A plain Java class used to represent data without requiring a special framework base class.
Why do we use getters and setters? They provide controlled access to private fields and establish conventional JavaBean property access.
What is serialization? Converting a Java object into a data format such as JSON for transport.

What is deserialization? Converting JSON or another serialized representation into a Java object.

How does Rest Assured serialize a POJO? It uses a supported object mapper, such as Jackson or Gson, available/configured in the project.

What is the purpose of @JsonProperty? It explicitly maps a Java property to a JSON property name, which helps with names such as "long".

What does Lombok @Data do? It generates common methods including getters, setters, toString, equals/hashCode, and a required-arguments constructor.

Why is @Data risky for a password DTO? Its generated toString() may include the password or other sensitive fields.

How do you represent nested JSON? Use a POJO field whose type is another POJO, such as ProductRequest containing Rating.

Why prefer BigDecimal for prices? It avoids many binary floating-point rounding issues common with float/double.

Does a logged HTTP response mean a test passed? No. Assertions must compare the actual response with the expected contract.

Should a create request include server-generated id? Usually no, unless the API contract explicitly requires client-supplied IDs.

Final checklist before you submit a POJO-based API test

- ☐ POJO fields match the API request schema.
- ☐ Fields use appropriate Java types.
- ☐ Getters follow the expected naming pattern or Lombok is correctly configured.
- ☐ Nested JSON is represented by nested POJO fields.
- ☐ JSON special names are mapped explicitly where needed.
- ☐ Rest Assured has a compatible object mapper available.
- ☐ Request Content-Type is correct.
- ☐ Test asserts status code and key response fields.
- ☐ Logging does not expose passwords, tokens, or personal data.
- ☐ Test data is synthetic and unique where necessary.
- ☐ The test expectation comes from the API contract, not from guesswork.

Key takeaway

POJOs make API tests easier to read and maintain because Java objects represent the same structure as the JSON contract. Learn the structure first, create a correctly typed model, populate it, serialize it through Rest Assured, and assert that the API behaves as documented.